

A Progressive Strategy for DNA Resequencing Problem

Chia-Wei Lu, Chuan Yi Tang and R. C. T. Lee

Department of Computer Science

National Tsing Hua University, Hsinchu, Taiwan R. O. C. (300)

d9762807@oz.nthu.edu.tw, cytang@cs.nthu.edu.tw, rctlee@ncnu.edu.tw

Abstract

In this paper, we are going to solve the DNA resequencing problem. We are given a set of subsequences obtained by the next generation sequencing (NGS) technology and we are asked to assemble them into a DNA sequence with the aid of a reference sequence. In this paper, we propose a progressive strategy to solve the resequencing problem. Our algorithm allows us to map data to reference sequence efficiently. Compared with other available tools, our approach allows us to be able to map much more subsequences to the reference sequence.

considers substitution. The Hamming distance between S_1 and S_2 is denoted as $HD(S_1, S_2)$.

In this paper, we shall use the Hamming distance and the approximate string matching problem is defined as follows: Given a text T , a pattern P and a prespecified error bound k , the approximate string matching problem is to find all substrings S 's in T such that $HD(S, P) \leq k$. Many approximate string algorithms have been developed [4, 5, 7, 13, 16 and 18].

1 A Brief Introduction of the String Matching Problems

As we shall see later, the DNA resequencing problem is a variation of the traditional string matching problems. Therefore, a brief introduction of the string matching problems and notations often used will be first given below. In string matching problems, there are a text $T = t_1 t_2 \cdots t_n$ and a pattern $P = p_1 p_2 \cdots p_m$ where both t_i 's and p_i 's are characters. We assume that $m \leq n$. We denote $t_i t_{i+1} \cdots t_j$ by $T(i, j)$ and $p_i p_{i+1} \cdots p_j$ by $P(i, j)$. We shall consider two different kinds of string matching problems: Exact string matching problem and approximate string matching problems.

For the exact string matching problem, we are given a text $T = t_1 t_2 \cdots t_n$ and a pattern $P = p_1 p_2 \cdots p_m$. Our job is to find whether P appears in T and if it does, where it appears. Many algorithms have been developed to solve this exact string matching problem [1-3 and 9].

For the approximate string matching problem, we first define two distance functions called edit distance and Hamming distance which measure the similarity between two strings.

- (1) Edit distance: The edit distance between two strings S_1 and S_2 is the minimum number of deletions, insertions and substitutions needed to transform S_2 to S_1 . The edit distance between S_1 and S_2 is denoted as $ED(S_1, S_2)$.
- (2) Hamming distance: The Hamming distance is a special version of edit distance which only

2 The Resequencing Problem

Our problem is a DNA sequencing problem. Suppose that we are given a DNA sequence T and we want to find its exact contents. This can be done by traditional methodologies. Unfortunately, it is very costly and time-consuming to complete the sequencing process because this DNA sequence is usually very long. For example there are probably $3 \cdot 10^9$ base pairs long in a human DNA sequence. A cheaper approach is to use an equipment to break down the sequence into small subsequences pieces which are called reads and then use some software to assembly these reads back to the original sequence.

In many applications, we do not really have to produce the original sequence. We are given a reference sequence R which is known to be similar to and we are asked to map these reads onto the reference sequence. The more reads are mapped onto R , the more successful we are because if a large number of reads are mapped back to R , we have almost constructed the original DNA sequence T .

Let us first have a description of our data. Our data are obtained by using the next generation sequencing (NGS) technology which cuts a DNA sequence T into many overlapping subsequences. NGS technologies, such as Illumina-Solexa, ABI-SOLiD and Roche-454 systems, can generate high throughput and short sequencing data. For example, Illumina-Solexa system can generate several millions reads of length 76 in a single run. These subsequences constitute the set $S = \{s_1, s_2, \dots, s_n\}$. The size of S is usually of 1 million to 10 millions.

As we mentioned before, we are given a reference DNA sequence R . We try to find locations of s_i in

R . Of course, we cannot expect s_i to appear exactly in R . Our resequencing problem is now defined as follows: We are given a set $S = \{s_1, s_2, \dots, s_n\}$ of short subsequences cut from an unknown DNA sequence T and a reference sequence R and we are asked to map s_i 's to R where s_i 's appear approximately in R . Note that we have not exactly defined the resequencing problem because the error bound k is not mentioned. We shall explain why this is so. The success of our algorithm is measured by how many s_i 's are mapped onto R .

After we map s_i 's to R approximately, we will change the contents of R . Let us give an example as shown in Figure 1 to illustrate this point. Suppose our reference sequence is $R = acactacgactacctccacg$ and $s_1 = atgattac$. Then we find a subsequence of R , namely $R(6, 13) = acgactac$ whose Hamming distance with respect to s_1 is 2. This is the subsequence of R whose Hamming distance with s_1 is the smallest. Suppose we have $s_2 = acaatatg$. Then, we find a substring of R , namely $R(1, 8) = acactacg$ whose Hamming distance with respect to s_2 is also 2. Suppose we have $s_3 = atatgatt$. Then we can map s_1 , s_2 and s_3 to R as follows:



Figure 1. A mapping result

After obtaining the results, we change $R(1, 13) = acactacgactac$ to $R(1, 13) = acaatatgattac$. We shall explain why we do this changing. To some degree, we are actually trying to reconstruct the original sequence T .

We can now see that to solve our problem, we must be able to solve the multiple approximate string matching problem. In the following section, we shall give an introduction of some major multiple approximate string matching algorithms.

3 Algorithms for the Multiple Approximate String Matching Problem

The multiple approximate string matching problem is defined as follows: We are given a set of patterns $S = \{s_1, s_2, \dots, s_n\}$, a sequence R and an error bound k . For every s_i , we are required to find all

subsequences in R whose Hamming distance with s_i are smaller than or equal to k . Algorithms to solve this problem can be categorized into two approaches:

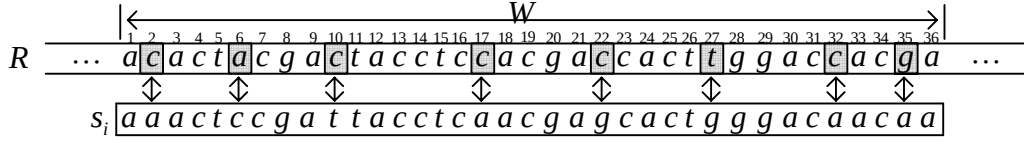
- Approach 1: Candidate Finding and Checking Approach

The main idea of this approach is to transform the approximate string matching problem into the exact string matching problem. For example, if $s_i = aacggatat$ and $k = 2$. We then first divide s_i into $k+1=3$ non-overlapping parts, `aac`, `gga` and `tat`, called seeds. If there is a substring $R(i, j)$ in R such that $HD(R(i, j), s_i) \leq 2$, there must be at least one seed which is exactly equal to a part of $R(i, j)$. This property was first pointed out in [18] and used in many approximate string matching algorithms [8, 10, 11 and 14-18]. Therefore, for any region in R , if there does not exist any exact occurrence of any seed in it, we may ignore this region. If there is a seed x exactly appearing in R , denote the corresponding substring of x in R to be y . Let us denote $R(i, j)$ to be the substring in R containing y , x and y are exactly aligned and the length of $R(i, j)$ is the same as that of s_i . We then check whether $HD(R(i, j), s_i) \leq 2$ or not. For example, if $s_i = aacggatat$, $k = 2$ and the seed 'tat' exactly appears in $R(17, 19)$. Suppose $R(11, 19) = aaagggtat$. Then we check $R(11, 19)$ and find that $HD(R(11, 19), s_i) = 2$. Consider $R(22, 30) = acacgtaag$. We can see that none of the seeds exactly appears in it. We may now ignore this region.

Many resequencing tools, such as SOAP [10], MAQ [11], SeqMap [8], RMap [17], and ZOOM [14], one way or the other, are based upon this approach.

- Approach 2: Error Patterns Generating and Searching Approach

In this approach, given an s_i , we first generate all error patterns whose distances from s_i are all smaller than or equal to the error bound k . For example, if $s_i = aacgt$ and $k = 2$, the pattern $p_1 = cacgg$ is an error pattern of s_i . Then we check whether $p_1 = cacgg$ exactly appears in R . If it does, we know that there is a substring $R(i, j)$ such that $HD(R(i, j), s_i) \leq k = 2$. Therefore, if we exhaustively generate all the error patterns of s_i and search the exact occurrences of error patterns in R , we can find all of the approximate occurrences of s_i in R . The Bowtie [12] and [6] algorithms are based upon this approach.

Figure 2. An s_i mapping onto a high variant region of R

4 The Error Bound Setting Problem

In the above, we indicated that we have to solve the multiple approximate string matching problem. In Figure 2, we show a subsequence W in R and a subsequence s_i .

As can be seen, if we set the error bound to be 8, s_i can be mapped and it cannot be mapped if the error bound is set to be smaller than 8. From this example, we may conclude that the error bound cannot be set to be too small. If the error bound is too small, there will not be many solutions to the approximate string matching problem. Thus many s_i 's will not be mapped to R and therefore we fail to achieve our mission.

But, we shall show that a large error bound is not practical either. Let us first consider Approach 1 introduced in Section 3. If this approach is used, we will cut every s_i into $k+1$ seeds. Since k is large, each seed is small and therefore will appear in many locations in R . This means that we cannot ignore many regions. Thus this approach will be very time-consuming.

If we use Approach 2, when k is large, the number of error sequences will be exceedingly large, and the cost for searching error sequences will also be very high. This will make the approach even more not practical. For instance, for one read with length 35, if $k=8$ there will be roughly 1.5×10^{12} error patterns.

We are thus in a dilemma. The error bound can be neither too large, nor too small. In the next section, we propose our progressive algorithm to solve our problem which uses low error bound and yet still can map many reads onto the reference sequence.

5 Our Progressive Strategy to Solve the Resequencing Problem

Our progressive approach uses a small error bound k . It can be illustrated by considering the following example.

As can be seen in Figure 3, since k is small, no reads are mapped to $W(14, 26)$. Thus, we do not know how to handle this region. Since $W(1, 13)$ is covered by many reads, $w_2 = c$ is mapped by many different reads. We therefore conduct a voting to change w_2 from c to a . Similarly, w_6, w_{10}, w_{32} and w_{35} will be modified to be t, g, a and c respectively. In this case, we do not modify w_{27} to be 'a' because there are not many reads mapped to this location. We use a conservative voting algorithm to modify the reference sequence R , and it will be described later.

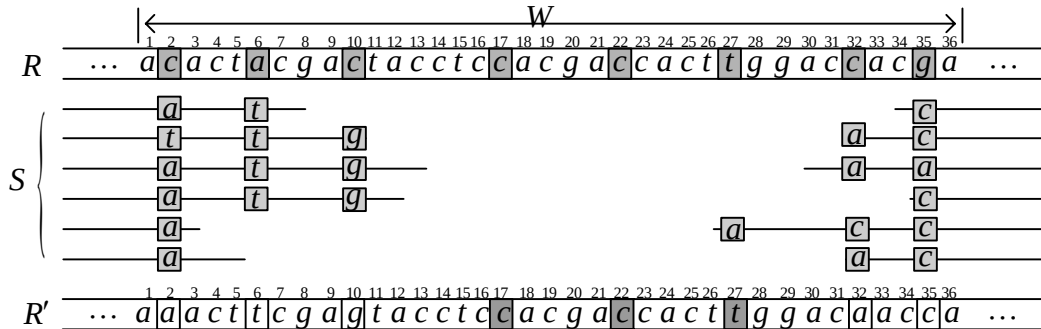
In the following, we give our progressive approach to solve the resequencing problem. We first give an explanation of some terms.

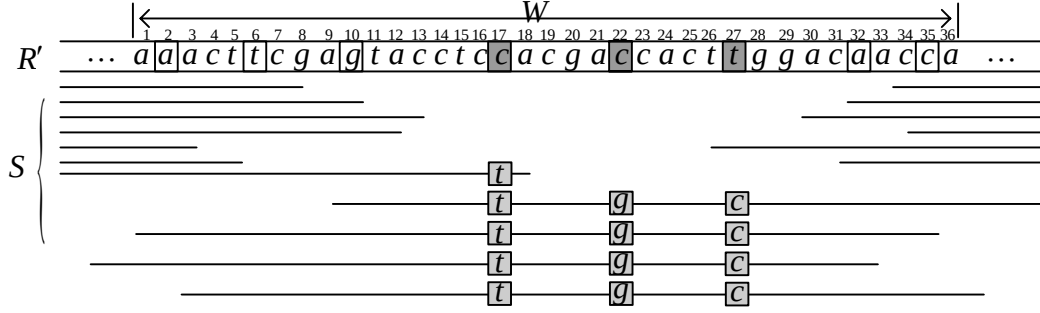
1. Let $s_i[j]$ be the j -th character counting from the left side of the read s_i , and for every $s_i[j]$, there is a quality score associated with it given by the NGS system.

2. For every location i of R , it may be mapped by any character of the alphabet set. Let x denote a character in the alphabet set. $R[i].score[x]$ is an indication of how many times x is mapped to $R[i]$.

Step 1: Mapping

Let k be the average number of sequencing errors in S caused by the NGS system which we use to obtain sequences. We set $k = k + 1$. For example, suppose there are 5% sequencing errors in

Figure 3. A mapping results with small k and modifications on R

Figure 4. A mapping results to the modified reference sequence R'

the reads with length 35. We then use $k = 5\% \times 35 + 1 \approx 2 + 1 = 3$. For each s_i of S , find the locations where it occurs in R with error k . In this step, any approximate string matching algorithm can be used to map s_i 's to R . In our case, we used candidate finding and checking approach [18].

Step 2: Voting

If a read s_i of length m is mapped to $R(i, i + m - 1)$, and the quality score (obtained from *.fq file) of $s_i[j]$ is larger than a preset value $qual_Threshold$, we add $R[i + j].score[s_i[j]]$ one score.

Step 3: Modifying Reference Sequence

For every location i of R , let $TotalScore = \sum R[i].score[q]$, for all $q \in \Sigma$. Let $depth_Threshold$ and $Sratio$ be two parameters set by us. Then, for every $R[i]$ of R , we modify it to be x if all the following conditions are satisfied. and denote the modified R to be R' .

Condition 1:

$$R[i].score[x] \geq depth_Threshold.$$

Condition 2:

$$R[i].score[x] \geq TotalScore * Sratio,$$

where $0 \leq Sratio \leq 1$.

Step 4: Iteration

If the number of reads being successful mapped onto the modified reference sequence is large enough, terminate the process; otherwise, set $R = R'$ and start another round from Step 1.

The important point is that we do not stop in Step 3. We will start another round of mapping reads to the modified reference sequence R' after the

sequence R is modified. Note that we are using a small error bound. Therefore, there are many regions in R onto which no reads are mapped. This iteration step can reduce the number of regions where no reads are mapped onto.

For example, consider Figure 4. We map the reads to R' which is a modified reference sequence from R in Figure 3, and now $W(14, 26)$ can be mapped with $k = 3$ by many reads.

6 Experiments

We used the genome sequencing data of *Helicobacter pylori* which were generated by Solexa system. There were 3,074,139 reads and each was of length 76. We selected the genome sequence “*Helicobacter pylori* 26695” to be our reference sequence R , downloaded from NCBI, and its length is around 1.6×10^6 . Since elements located at right side (3-end) of Solexa's reads are usually with low-quality, we trimmed the right most side 10 characters of every read. We first show the performance of three tools, Bowtie, SOAP and MAQ by using our data on sever platform. In Table 1, we list the processing time, percentage of mapped reads, and percentage of the number of $R[i]$ which is mapped by at least one read (% of genome covered). The experimental results in Table 1 show that there are nearly 60% unmapped reads with $k = 3$, and about 40% while $k = 5$.

In Table 2, we show the results of 1~10-th iterations of our algorithm with $k = 3$, $qual_Threshold = 70$, $depth_Threshold = 3$ and $Sratio = 0.9$.

Table 1. The performance of some available tools

	Bowtie, $k = 3$	SOAP, $k = 3$	SOAP, $k = 5$	MAQ, $k = 5$
Time (sec)	680	63	79	160
Mapped reads (%)	41.98	42.55	58.39	58.89
Genome covered (%)	78.44	79.53	86.31	87.27

Table 2. The results of 1~10-th iterations of our algorithm with $k = 3$

i -th iteration	Time (sec)	Mapped reads (%)	Genome covered (%)	i -th iteration	Time (sec)	Mapped reads (%)	Genome covered (%)
1-th	44	46.81	80.82	6-th	52	72.36	87.57
2-th	50	65.11	84.35	7-th	52	72.63	87.78
3-th	51	69.61	85.99	8-th	53	72.80	87.93
4-th	52	71.22	86.81	9-th	52	72.94	88.04
5-th	52	71.95	87.28	10-th	53	73.02	88.12

From the results of Table 2, we can see that there are nearly 30% unmapped reads after 3 iterations and 27% after 10 iterations with $k = 3$. Thus our approach is much more effective than all of the other existing software tools.

7 Conclusions

We propose a progressive algorithm for the resequencing problem. We use a small error bound and can efficiently map more reads to a reference sequence. Besides, the mapping results of our algorithm are with high confidence because we always use a small error bound.

References

- [1] Boyer, R.S. and Moore, J.S., A fast string searching algorithm, *Communications of the ACM*, Vol. 20, 1977, pp. 762-772.
- [2] M. Crochemore, C. Hancart and T. Lecroq, *Algorithms on Strings*, Cambridge University Press, 2007, 383 pages, ISBN 978-0-521-84899-2.
- [3] M. Crochemore and W. Rytter, *Jewels of Stringology*, World Scientific, 2002, 310 pages, ISBN 981-02-4782-6.
- [4] K. Fredriksson and G. Navarro, Average-Optimal Multiple Approximate String Matching, *ACM Journal of Experimental Algorithmics*, Vol. 9, No. 1.4, 2004, pp. 1-47.
- [5] T. N. D. Huynh, W. K. Hon, T. W. Lam and W. K. Sung, Approximate String Matching Using Compressed Suffix Arrays, *Theoretical Computer Science*, Vol. 352, 2006, pp. 240-249.
- [6] Wing-Kai Hon, Tak Wah Lam, Kunihiro Sadakane, Wing-Kin Sung, Siu-Ming Yiu, A Space and Time Efficient Algorithm for Constructing Compressed Suffix Arrays, *Algorithmica*, Vol. 48, 2007, pp. 23-36.
- [7] H. Hyrö and G. Navarro, Bit-parallel Witnesses and their Applications to Approximate String Matching, *Algorithmica*, Vol. 41, No. 3, 2005, pp. 203-231.
- [8] Hui Jiang and Wing Hung Wong, SeqMap: mapping massive amount of oligonucleotides to the genome, *Bioinformatics*, Vol. 24, 2008, pp. 2395-2396.
- [9] Knuth, D.E., Morris (Jr), J.H. and Pratt, V.R., Fast pattern matching in strings, *SIAM Journal on Computing*, Vol. 6, No. 2, 1977, pp. 323-350.
- [10] Ruiqiang Li, Yingrui Li, Karsten Kristiansen and Jun Wang, SOAP: short oligonucleotide alignment program, *BIOINFORMATICS APPLICATIONS NOTE*, Vol. 24, no. 5, 2008, pp. 713-714.
- [11] Heng Li, Jue Ruan and Richard Durbin, Mapping short DNA sequencing reads and calling variants using mapping quality scores, *Genome Research*, Vol. 18, 2008, pp. 1851-1858.
- [12] Ben Langmead, Cole Trapnell, Mihai Pop and Steven L. Salzberg, Ultrafast and memory-efficient alignment of short DNA sequences to the human genome, *Genome Biol*, Vol. 10, 2009, pp. R25.
- [13] G. Landau and U. Vishkin, Fast Parallel and Serial Approximate String Matching, *Journal of Algorithms*, Vol. 10, 1989, pp. 157-169.
- [14] Hao Lin, Zefeng Zhang, Michael Q. Zhang, Bin Ma and Ming Li, ZOOM! Zillions of oligos mapped, *Bioinformatics*, Vol. 24, 2008, pp. 2431-2437.
- [15] G. Navarro and R. Baeza-Yates, Very fast and simple approximate string matching, *Information Processing Letters*, Vol. 72, 1999, pp. 65-70.
- [16] G. Navarro and R. Baeza-Yates, A Hybrid Indexing Method for Approximate String Matching, *Journal of Discrete Algorithms*, No. 1, Vol. 1, 2000, pp. 205-239.
- [17] Andrew D. Smith, Zhenyu Xuan and Michael Q. Zhang, Using quality scores and longer reads improves accuracy of Solexa read mapping, *BMC Bioinformatics*, Vol. 9, 2008, pp. 128.
- [18] S. Wu and U. Manber, Fast Text Searching: Allowing Errors, *Communications of the ACM*, Vol. 35, 1992, pp. 83-91.